

Module 6: Compute

1. Compute Services Overview

Services	Key Concepts	Characteristics	Ease of Use
Amazon EC2	- Infrastructure as a service (IaaS) - Instance-based - Virtual machines	Provision virtual machines that you can manage as you choose	A familiar concept to many IT professionals.
AWS Lambda	- Serverless computing - Function-based - Low-cost	- Write and deploy code that executes on a schedule or that can be triggered by events - Use when possible (architect for the cloud)	A relatively new concept for many IT staff members, but easy to use after you learn how.
- Amazon ECS - Amazon EKS - AWS Fargate - Amazon ECR	- Container-based computing - Instance-based	Spin up and execute jobs more quickly	AWS Fargate reduces administrative overhead, but you can use options that give you more control.
AWS Elastic Beanstalk	- Platform as a service (PaaS) - For web applications	- Focus on your code (building your application) - Can easily tie into other services—databases, Domain Name System (DNS), etc.	Fast and easy to get started.

2. Amazon EC2 Part 1



- Amazon Elastic Compute Cloud (Amazon EC2)
 - Provides virtual machines—referred to as EC2 instances—in the cloud.
 - Gives you full control over the guest operating system (Windows or Linux) on each instance.
- You can launch instances of any size into an Availability Zone anywhere in the world.
 - Launch instances from Amazon Machine Images (AMIs).
 - Launch instances with a few clicks or a line of code, and they are ready in minutes.
- You can control traffic to and from instances.

1. Select an AMI

Choices made using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair



- Amazon Machine Image (AMI)
 - Is a template that is used to create an EC2 instance (which is a **virtual machine, or VM**, that runs in the AWS Cloud)
 - Contains a **Windows** or **Linux** operating system
 - Often also has some **software** pre-installed
- AMI choices:
 - Quick Start – *Linux and Windows AMIs that are provided by AWS*
 - My AMIs – *Any AMIs that you created*
 - AWS Marketplace – *Pre-configured templates from third parties* 
 - Community AMIs – *AMIs shared by others; use at your own risk*

2. Select an instance type

Choices made using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- Consider your use case
 - How will the EC2 instance you create be used?
- The **instance type** that you choose determines –
 - **Memory** (RAM)
 - **Processing power** (CPU)
 - **Disk space and disk type** (Storage)
 - **Network performance**
- Instance type categories –
 - General purpose
 - Compute optimized
 - Memory optimized
 - Storage optimized
 - Accelerated computing
- Instance types offer *family, generation, and size*



Select instance type: Based on use case

	 General Purpose	 Compute Optimized	 Memory Optimized	 Accelerated Computing	 Storage Optimized
Instance Types	a1, m4, m5, t2, t3	c4, c5	r4, r5, x1, z1	f1, g3, g4, p2, p3	d2, h1, i3
Use Case	Broad	High performance	In-memory databases	Machine learning	Distributed file systems

3. Amazon EC2 Part 2

3. Specify network settings

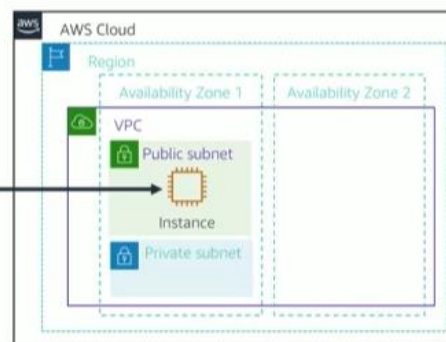


Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- Where should the instance be deployed?
 - Identify the VPC and optionally the **subnet**
- Should a **public IP address** be automatically assigned?
 - To make it internet-accessible

Example: specify to deploy the instance here



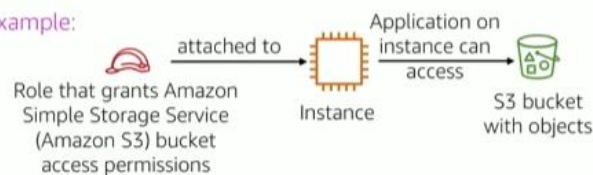
4. Attach IAM role (optional)

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- Will software on the EC2 instance need to interact with other AWS services?
 - If yes, attach an appropriate **IAM Role**.
- An AWS Identity and Access Management (IAM) role that is attached to an EC2 instance is kept in an **instance profile**.
- You are *not* restricted to attaching a role only at instance launch.
 - You can also attach a role to an instance that already exists.

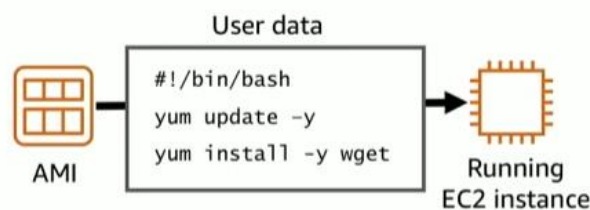
Example:



5. User data script (optional)

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair



- Optionally specify a user data script at instance launch
- Use **user data** scripts to customize the runtime environment of your instance
 - Script executes the **first time the instance starts**
- Can be used strategically
 - For example, reduce the number of custom AMIs that you build and maintain

6. Specify storage

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. **Storage options**
7. Tags
8. Security group
9. Key pair

- Configure the **root volume**
 - Where the guest operating system is installed
- Attach **additional storage volumes** (optional)
 - AMI might already include more than one volume
- For each volume, specify:
 - The **size** of the disk (in GB)
 - The **volume type**
 - Different types of solid state drives (SSDs) and hard disk drives (HDDs) are available
 - If the volume will be deleted when the instance is terminated
 - If **encryption** should be used



Amazon EC2 storage options

- **Amazon Elastic Block Store (Amazon EBS)** –
 - **Durable**, block-level storage volumes.
 - You can stop the instance and start it again, and the data will still be there.
- **Amazon EC2 Instance Store** –
 - Storage is provided on disks that are attached to the host computer where the EC2 instance is running.
 - **If the instance stops, data stored here is deleted.**
- Other options for storage (not for the root volume) –
 - Mount an **Amazon Elastic File System (Amazon EFS)** file system.
 - Connect to **Amazon Simple Storage Service (Amazon S3)**.

4. Amazon EC2 Part 3

7. Add tags

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- A **tag** is a label that you can assign to an AWS resource.
 - Consists of a *key* and an optional *value*.
- Tagging is how you can attach **metadata** to an EC2 instance.
- Potential benefits of tagging—Filtering, automation, cost allocation, and access control.

Example:

Key	Value
Name	WebServer1

[Add another tag](#) (Up to 50 tags maximum)

8. Security group settings

Choices made by using the Launch Instance Wizard:

1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- A **security group** is a **set of firewall rules** that control traffic to the instance.
 - It exists *outside* of the instance's guest OS.
- Create **rules** that specify the **source** and which **ports** that network communications can use.
 - Specify the **port** number and the **protocol**, such as Transmission Control Protocol (TCP), User Datagram Protocol (UDP), or Internet Control Message Protocol (ICMP).
 - Specify the **source** (for example, an IP address or another security group) that is allowed to use the rule.

Type	Protocol	Port Range	Source
SSH	TCP	22	My IP 72.21.198.67/32

9. Identify or create the key pair

Choices made by using the Launch Instance Wizard:

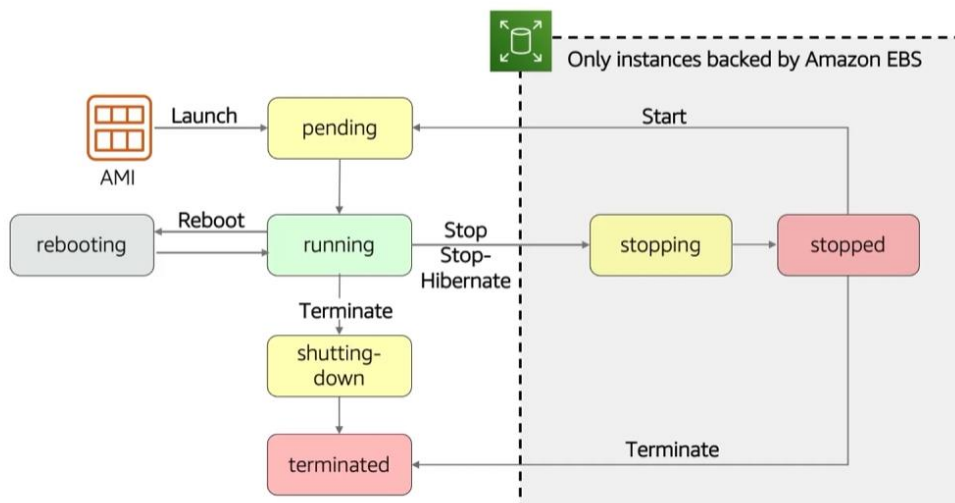
1. AMI
2. Instance Type
3. Network settings
4. IAM role
5. User data
6. Storage options
7. Tags
8. Security group
9. Key pair

- At instance launch, you specify an existing key pair or create a new key pair.
- A **key pair** consists of –
 - A **public key** that AWS stores.
 - A **private key** file that you store.
- It enables secure connections to the instance.
- For Windows AMIs –
 - Use the private key to obtain the administrator password that you need to log in to your instance.
- For Linux AMIs –
 - Use the private key to use SSH to securely connect to your instance.

mykey.pem



Amazon EC2 instance lifecycle



5. Amazon EC2 Cost Optimization

Amazon EC2 pricing models



On-Demand Instances

- Pay by the hour
- No long-term commitments.
- Eligible for the [AWS Free Tier](#).

Dedicated Hosts

- A physical server with EC2 instance capacity fully dedicated to your use.

Dedicated Instances

- Instances that run in a VPC on hardware that is dedicated to a single customer.

Reserved Instances

- Full, partial, or no upfront payment for instance you reserve.
- Discount on hourly charge for that instance.
- 1-year or 3-year term.

Scheduled Reserved Instances

- Purchase a capacity reservation that is always available on a recurring schedule you specify.
- 1-year term.

Spot Instances

- Instances run as long as they are available and your bid is above the Spot Instance price.
- They can be interrupted by AWS with a 2-minute notification.
- Interruption options include terminated, stopped or hibernated.
- Prices can be significantly less expensive compared to On-Demand Instances
- Good choice when you have flexibility in when your applications can run.

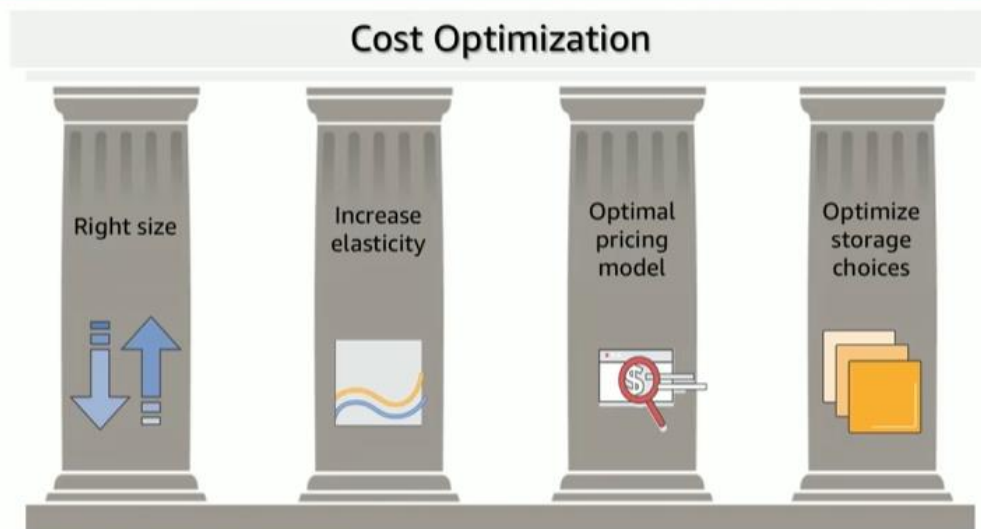
Per second billing available for On-Demand Instances, Reserved Instances, and Spot Instances that run Amazon Linux or Ubuntu.

Benefits:

On-Demand Instances	Spot Instances	Reserved Instances	Dedicated Hosts
• Low cost and flexibility	• Large scale, dynamic workload	• Predictability ensures compute capacity is available when needed	• Save money on licensing costs • Help meet compliance and regulatory requirements

Use cases:

On-Demand Instances	Spot Instances	Reserved Instances	Dedicated Hosts
• Short-term, spiky, or unpredictable workloads • Application development or testing	• Applications with flexible start and end times • Applications only feasible at very low compute prices • Users with urgent computing needs for large amounts of additional capacity	• Steady state or predictable usage workloads • Applications that require reserved capacity, including disaster recovery • Users able to make upfront payments to reduce total computing costs even further	• Bring your own license (BYOL) • Compliance and regulatory restrictions • Usage and licensing tracking • Control instance placement



© 2019 Amazon Web Services, Inc. or its Affiliates. All rights reserved.

49

Right-size

Choose the right balance of instance types. Notice when servers can be either sized down or turned off, and still meet your performance requirements.

Increase elasticity

Design your deployments to reduce the amount of server capacity that is idle by implementing deployments that are elastic, such as deployments that use automatic scaling to handle peak loads.

Optimal pricing model

Recognize the available pricing options. Analyse your usage patterns so that you can run EC2 instances with the right mix of pricing options.

Optimize storage choices

Analyse the storage requirements of your deployments. Reduce unused storage overhead when possible, and choose less expensive storage options if they can still meet your requirements for storage performance.

6. Container Services

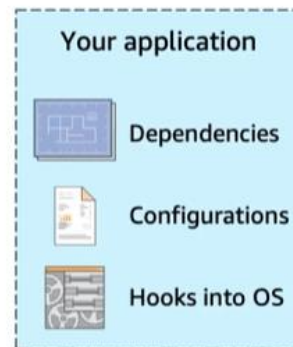
Container basics

- Containers are a method of operating system virtualization.

- Benefits –

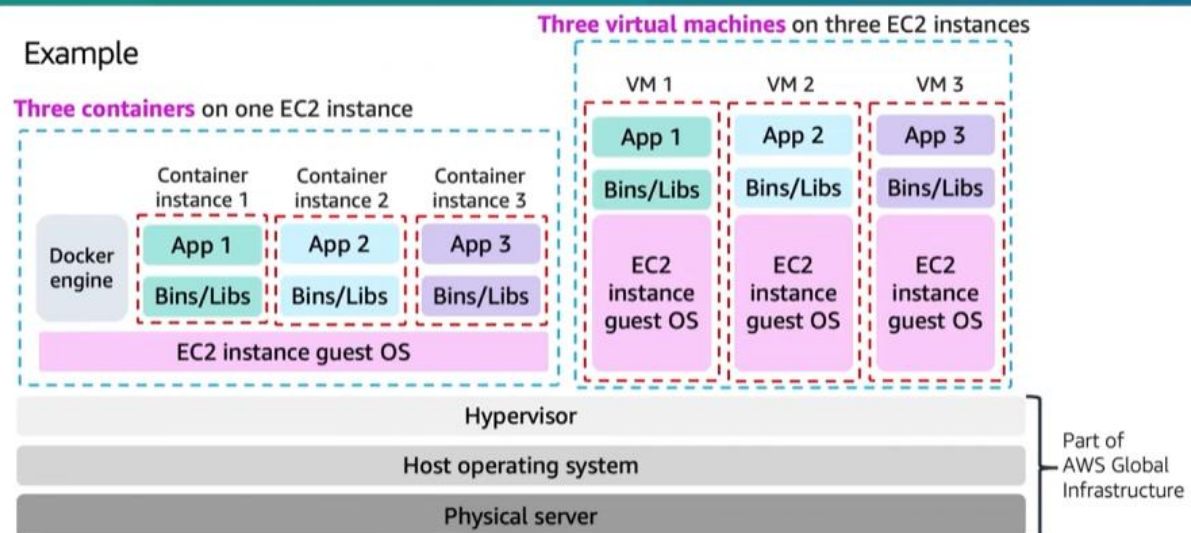
- Repeatable.
- Self-contained environments.
- Software runs the same in different environments.
 - Developer's laptop, test, production.
- Faster to launch and stop or terminate than virtual machines

Your Container



Containers versus virtual machines

Example



Amazon Elastic Container Service (Amazon ECS)



- Amazon Elastic Container Service (**Amazon ECS**) –
 - A highly scalable, fast, **container management service**
- Key benefits –
 - Orchestrates the running of Docker containers
 - Maintains and scales the fleet of nodes that run your containers
 - Removes the complexity of standing up the infrastructure
- Integrated with features that are familiar to Amazon EC2 service users –
 - Elastic Load Balancing
 - Amazon EC2 security groups
 - Amazon EBS volumes
 - IAM roles



Amazon Elastic Container Service

Amazon Elastic Kubernetes Service (Amazon EKS)



- Amazon Elastic Kubernetes Service (**Amazon EKS**)
 - Enables you to run Kubernetes on AWS
 - Certified Kubernetes conformant (supports easy migration)
 - Supports Linux and Windows containers
 - Compatible with Kubernetes community tools and supports popular Kubernetes add-ons
- Use Amazon EKS to –
 - Manage clusters of Amazon EC2 compute instances
 - Run containers that are orchestrated by Kubernetes on those instances



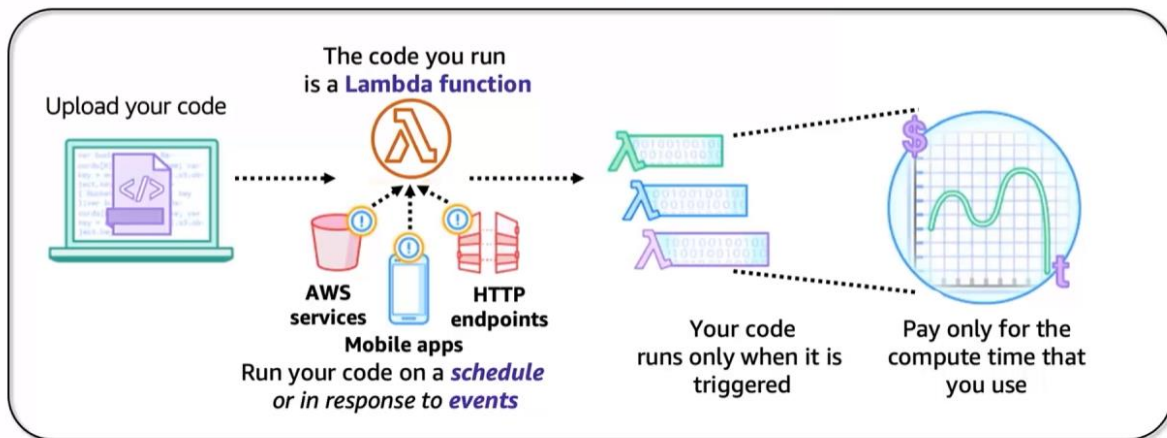
Amazon Elastic Kubernetes Service

7. Introduction to AWS Lambda

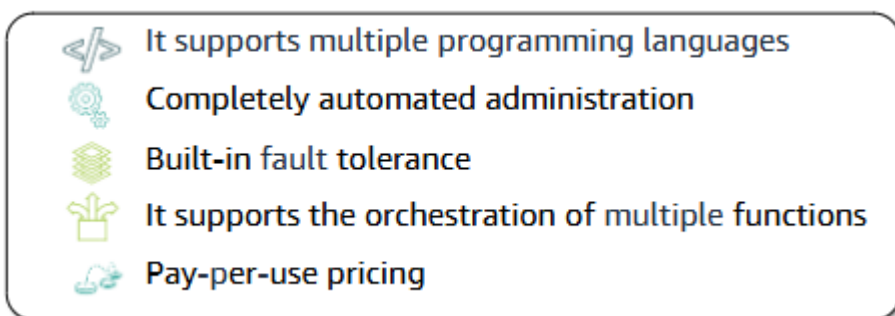
AWS Lambda: Run code without servers



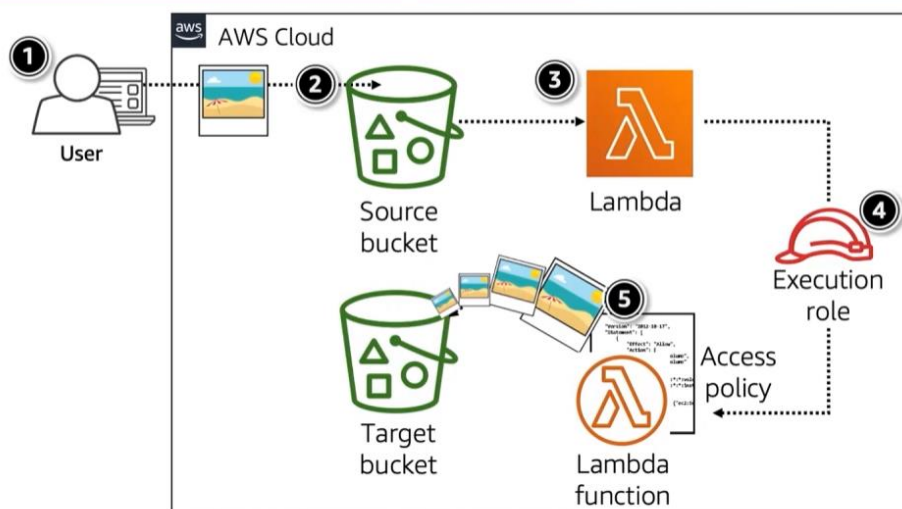
AWS Lambda is a **serverless** compute service.



Benefits:



Event-based Lambda function example: Create thumbnail images



- 1: A user uploads an object to the source bucket in Amazon S3 (object-created event).
- 2: Amazon S3 detects the object-created event.

3: Amazon S3 publishes the object-created event to Lambda by invoking the Lambda function and passing event data.

4: Lambda runs the Lambda function by assuming the execution role that you specified when you created the Lambda function.

5: Based the event data that the Lambda function receives, it knows the source bucket name and object key name. The Lambda function reads the object and creates a thumbnail by using graphics libraries, and saves the thumbnail to the target bucket

AWS Lambda quotas

Soft limits per Region:

- Concurrent executions = 1,000
- Function and layer storage = 75 GB

Hard limits for individual functions:

- Maximum function memory allocation = 10,240 MB
- Function timeout = 15 minutes
- Deployment package size = 250 MB unzipped, including layers
- Container image code package size = 10 GB

8. Introduction to AWS Elastic Beanstalk

AWS Elastic Beanstalk



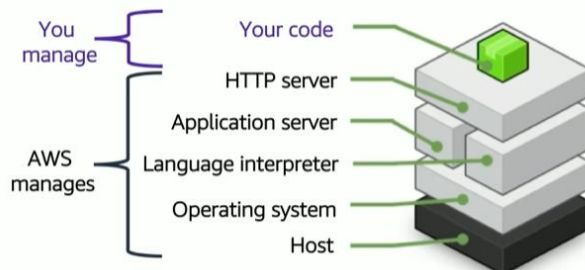
AWS Elastic
Beanstalk

- An easy way to get **web applications** up and running
- A **managed service** that automatically handles –
 - Infrastructure provisioning and configuration
 - Deployment
 - Load balancing
 - Automatic scaling
 - Health monitoring
 - Analysis and debugging
 - Logging
- No additional charge for Elastic Beanstalk
 - Pay only for the underlying resources that are used

AWS Elastic Beanstalk deployments



- It supports web applications written for common platforms
 - Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker
- You upload your code
 - Elastic Beanstalk automatically handles the deployment
 - Deploys on servers such as Apache, NGINX, Passenger, Puma, and Microsoft Internet Information Services (IIS)



Benefits:



Fast and simple
to start using



Developer
productivity



Difficult to
outgrow



Complete resource
control